

Capture, Verify, Govern: Decision Provenance for Agent Artifact Handoffs

Anonymous Authors

Abstract—When one agent continues another’s work, it inherits decisions as well as files. A recorded decision need not be supported by its evidence, and a supported decision need not be authorized for reuse. We present a capture–verify–govern framework that separates these responsibilities and specifies the information needed at each boundary. TRACE records submitted decisions, a domain-specific verifier checks their evidential consistency, and Proofpress separates evidence intake from claim review and governed reuse. We instantiate the verification layer on RSI-Exam program-version selection. In twelve synthetic packages, format checks reject one of nine authored faults, file-binding checks reject three, and full checks reject nine; two valid controls and a consistent unsigned reward rewrite pass. A separate local integration demonstration imports three TRACE decisions as evidence without creating claims or admissions; even a verifier-rejected numerical error remains importable as evidence. These results characterize verification and intake boundaries, not the effectiveness of a complete governance workflow. Recording is not verification, consistency is not authenticity, and neither grants permission to reuse.

Index Terms—decision provenance, agent handoffs, verification, governed reuse

I. INTRODUCTION

Suppose one agent hands a revised program to another agent or researcher. The recipient can inspect the files, but also needs to know why this version was selected and whether it is appropriate to rely on that decision now. A tool-call log, a matching file hash, and a reuse authorization answer different questions. Treating any one as a blanket trust signal obscures what the recipient can actually check.

Our running example is an AI coding agent improving a program that plays 2048. It edits, tests, and keeps a version or returns to an earlier one. RSI-Exam provides this setting: agents improve task programs using visible evaluations, then submit them for grading on unseen data [1]. Here, self-improving means improving the task program, not the model’s weights. A kept version can become the parent of later edits. Its successor inherits the consequences of that choice without necessarily retaining the producing runtime.

We ask: *how should a submitted decision become checkable evidence, and what must remain separate before downstream reuse?* The audit unit is an explicit decision and its supporting artifacts, not every tool call or inferred private rationale. We propose three responsibilities: *capture* the submitted decision; *verify* its domain-specific evidence relationships; and *govern* whether a bounded claim may enter a recipient’s context. No stage automatically confers the authority of the next.

Our contributions are (1) this separation and an evidence contract for crossing its boundaries; (2) a reference instantia-

tion using TRACE, an RSI decision verifier, and Proofpress; and (3) a controlled characterization of the verifier plus a local demonstration of evidence-only intake. The framework is a design contribution, not a claim that its full workflow has been empirically validated. We test packages and local interfaces, not a multi-agent network, human-review outcomes, or improvements in downstream performance.

A. A real record shows the evidence gap

A retained record from an actual RSI run makes the problem concrete. A candidate reports a higher visible mean score than its parent, 48115 versus 38974.5, but is marked reverted without a recorded evaluation under the selection rule. The submitted version instead branches from that parent. The higher reported score alone cannot explain this choice, nor does it prove that reverting was wrong.

We can verify the retained experiment log’s hash and recover the declared branch history. But original code snapshots and per-seed measurements are absent from this retained export, so we cannot check their bytes or recalculate the decision’s statistical support. The companion’s Case R inspection reports these limits explicitly. This real-run example motivates what must be captured; the controlled experiment below tests the verifier on a different, fully supplied synthetic package.

II. THE CAPTURE–VERIFY–GOVERN FRAMEWORK

A. Responsibilities and the evidence contract

The producer may omit decisions, supply mismatched artifacts, or coherently fabricate an entire package. The receiver has the disclosed bytes, a chosen checker, and its own reuse policy; it has no trusted witness to execution. The framework makes these limits visible instead of interpreting a successful import or check as general trust.

At the **capture boundary**, the transferable record needs a stable decision identity, proposer and resolver identities, disposition, revision links, artifact references and the rule being claimed. Unsubmitted choices remain outside the record. TRACE represents the decision; a task profile supplies the meaning of its evidence and rule. A recorded resolver identity is attribution, not authenticated authority.

At the **verification boundary**, a reusable check result must identify the checked package, checker/profile revision, outcomes, diagnostics and coverage limits. A scalar “pass” without those bindings cannot say which decision was checked. Our RSI package retains the rule and raw measurements needed to recompute it; the ARA companion binds sources and reported

TABLE I
SEPARATE OUTPUTS AND AUTHORITY BOUNDARIES. COMPONENT NAMES
IDENTIFY THIS INSTANTIATION, NOT MANDATORY PRODUCTS.

Stage	Output and what it does not establish
Capture	TRACE decision: identities, actors, disposition, rationale and revision/evidence references. Not proof of correctness or complete capture.
Verify	Domain checker: outcomes and diagnostics tied to a supplied package and check scope. Not proof of authentic execution.
Govern	Proofpress: an evidence-bound candidate, separate review, and eligibility for scoped reuse. Not a substitute for domain checks.

outcomes. These checks establish *decision consistency*, not that an execution produced the measurements.

At the **governance boundary**, the candidate must state a bounded claim and intended reuse scope, with references to its evidence and any check results. An authorized human’s admission decision is distinct from the producing agent’s proposal and from a check result. The agent cannot approve its own claim. A downstream consumer should retrieve only admitted, current, in-scope, actor-eligible context; admission is not permanent proof of truth.

This contract is a composition rule, not a new universal wire schema. Capture, checking, and admission may be implemented by different systems. Their statuses must not be silently promoted: TRACE accepted, RSI keep, and Proofpress admission have different meanings.

B. Reference implementation and its present boundary

The RSI decision gate records a proposing agent and a *system* resolver. A converter maps keep/revert/pending and confirmation links into TRACE 0.5.1. It checks record-level constraints but does not recompute the measurements. Separately, the offline verifier consumes the full capsule and files. TRACE is a representation of submitted decisions, not a replacement for that package.

Proofpress imports a bounded projection of TRACE as external evidence. Within a confidence block it retains interval, method name/resample count, sample size and named evidence digests; it drops other structured rule fields such as estimate, threshold and bootstrap seed. It neither opens the referenced result files nor consumes a verifier success as an admission instruction. Claim proposal, review and context retrieval are separate operations.

Thus this is not yet one automatically enforced pipeline from verified package to approved context. In particular, the TRACE adapter does not require a receipt proving that the exact imported material passed the RSI checker. A full deployment must bind and check that receipt if its admission policy requires it. Our demonstration below tests this intake boundary and stops before human approval.

III. INSTANTIATION: RSI DECISION VERIFICATION

A. What the producer hands over

The evidence package contains saved code versions, test-result files, a decision log, submitted code, and a machine-readable summary called a *capsule*. The producer records decisions during the run and builds the capsule afterward. The recipient checks this directory without accessing the agent’s runtime or rerunning its program.

Each decision identifies five things:

- 1) The *parent* version being compared against and the new *candidate*.
- 2) The parent and candidate result files, labeled by role and bound by file hashes.
- 3) The score direction (higher or lower is better), statistic, confidence-interval procedure, random seed, and threshold.
- 4) The recorded calculation and decision: keep, revert, or await a confirmation evaluation.
- 5) A later confirmation decision, if one is required to resolve that pending state.

The capsule also links the submitted program to a recorded version. A version occurrence is distinct from its code contents: reverting can create another occurrence of the same code. Code identity follows the task’s staging rules, excluding Python bytecode caches; a separate whole-directory hash checks the supplied tree. These conventions avoid confusing a cache change with a program change.

B. A worked decision, from scores to submission

Table II follows the synthetic package used in our controlled experiment. v1 is the parent. Candidate v2 performs worse and is reverted. Candidate v3 is also derived from v1: its initial result is inconclusive, but a subsequent confirmation supports keeping it. The submitted program is v3.

TABLE II
RUNNING EXAMPLE FROM THE SYNTHETIC PACKAGE, NOT A REAL-RUN RESULT.
EACH ROW COMPARES THE CANDIDATE WITH v1.

Candidate/test	Mean delta	90% interval	Decision
v2 / initial	-318.75	[-382.5, -258.75]	Revert
v3 / initial	260	[-30, 583.75]	Pending
v3 / confirm	518.75	[467.5, 575]	Keep

The calculations use paired games: parent and candidate are evaluated on the same seed within each comparison. For a higher-is-better score, the mean improvement is

$$\hat{\Delta} = \frac{1}{n} \sum_{i=1}^n (s_{\text{candidate}}(i) - s_{\text{parent}}(i)).$$

The interval expresses uncertainty around the estimated improvement. The verifier recomputes the estimate and percentile-bootstrap interval from the supplied per-seed results and declared algorithm parameters. The example uses eight paired scores, 5000 resamples, and bootstrap seed 20260902. For a lower-is-better score the differences change sign.

In this package’s declared policy, an interval entirely below zero leads to revert, one entirely above zero supports keep, and an interval spanning zero requires confirmation. Thus the initial v3 result cannot by itself close the decision. Removing its confirmation while still submitting v3 leaves an unresolved pending decision. More generally, the verifier checks the recorded policy, not whether that policy is scientifically optimal.

C. What the recipient checks

The checks answer progressively stronger questions:

- 1) **Format:** Are required fields present and values well formed?
- 2) **File binding:** Do referenced files exist, and do their bytes match their declared hashes?
- 3) **Decision consistency:** Do parent/candidate references, recalculated statistics, decision rules, and submitted-version identity agree?

Full verification also reports *coverage*: whether every snapshot in the supplied version directory appears in the record. An unrecorded extra snapshot can leave the other checks passing but make coverage partial. Our experiment requires complete coverage for acceptance. This is completeness of the supplied directory, not proof that the producer disclosed every experiment.

Consider changing v2’s recorded mean delta from -318.75 to -317.75 , in both the decision log and capsule, then updating the log hash. Format and file binding still pass. Recalculation detects the discrepancy. This particular one-point error would not change the revert verdict; it tests numerical consistency, not the discovery of an incorrectly chosen winner. A separate test changes the submitted code while retaining the old version claim, exercising decision-to-artifact identity.

IV. EVALUATION AND INTEGRATION DEMONSTRATION

A. What is being tested?

We test the checking procedure, not the agent’s ability to play 2048. No new agent rollout is used for this experiment. We copy the same committed synthetic package twelve times and apply one specified change per case (or none for the clean control). Each copy must pass full verification before modification.

The twelve cases comprise nine deliberately introduced faults, two valid controls, and one authenticity-boundary control. Their manifest was committed before first execution, but after inspecting the implementation and development tests. This is an author-designed characterization, not an independent or held-out benchmark. The implementation is unchanged by the study.

We run three checking configurations on each package: **S** checks format only; **B** adds file bindings; **V** performs full verification and requires complete coverage. **B** uses the production hash functions but omits recalculation, decision rules, cross-field identity and coverage. These are internal ablations of our checker, not measurements of competing systems.

TABLE III
CONTROLLED PACKAGES. S: FORMAT; B: FORMAT + FILE HASHES; V: FULL CHECKS INCLUDING COVERAGE. A: ACCEPTS THAT CHECK SCOPE; R: REJECTS.

Package change	S	B	V
None (valid control)	A	A	A
Excluded caches differ; full hashes refreshed	A	A	A
Reward and declaration rewritten together	A	A	A
Unrecognized version status	R	R	R
Referenced measurement file removed	A	R	R
Recorded score direction reversed	A	A	R
Recorded mean delta increased by one	A	A	R
Parent/candidate measurements exchanged	A	A	R
Pending submission left unconfirmed	A	A	R
Submission changed; version claim retained	A	A	R
Recorded snapshot removed	A	R	R
Unrecorded snapshot added	A	A	R

To avoid testing only stale hashes, relevant decision changes are mirrored in the log and capsule and the log hash is refreshed. Swapping the parent/candidate measurements exchanges the referenced paths and hashes while retaining their role labels. Changing the submitted program also refreshes its hashes while retaining the claimed version. All cases, changed-file hashes and actual diagnostics are retained in the companion.

B. Results and interpretation

Table III reports every outcome. S rejects one of nine faults, B rejects three, and V rejects nine. The six additional refusals cover three measurement inconsistencies, an unresolved confirmation, a submitted-version mismatch, and an incomplete snapshot inventory. Thus files can match their declarations while the relationships needed to audit a decision still fail.

Both valid controls pass: the unchanged package and a package whose excluded bytecode caches differ. This establishes acceptance on these two examples, not a population false-positive rate. Likewise, the fault counts do not estimate detection rates on naturally occurring errors.

Rejection reasons matter. The extra snapshot is rejected for partial coverage, not an integrity error. The missing snapshot is correctly refused, but its diagnostic prefix differs from the manifest’s expected prefix. All nine faults are refused, but only eight match the prespecified diagnostic target. The unchanged expectation and actual output are both preserved; unexpected runtime exceptions abort the study rather than count as detections.

C. A passing rewrite defines the trust boundary

The final control changes an unsigned final reward file and the capsule’s matching reward to 0.5, then refreshes the file hash. V accepts. This is the final benchmark reward, not one of the paired visible-game measurements used for keep-or-revert decisions. The verifier checks agreement with the supplied reward file; it neither authenticates the grader nor recalculates that reward from underlying score details.

This accepted rewrite is important to the interpretation of the nine refusals. Our mechanism checks consistency among supplied declarations and bytes. A producer able to rewrite evidence consistently can still supply false material. Acceptance must therefore not be read as proof that these experiments really happened or that the agent is trustworthy.

D. Evidence intake does not grant reuse

In a separate local fixture demonstration, we check the clean RSI package, convert its three decisions to TRACE 0.5.1, and import them into a disposable Proofpress repository. The committed runner uses a pinned public Proofpress revision; it does not contact a hosted service or run an agent. The import creates three source records and three evidence items, but zero claims, zero admissions and an empty governed-context result. Re-import leaves the ledger unchanged. An unsupported version, malformed interval and changed normalized content under an existing identity are refused.

We then reuse the one-point mean-delta fault from the controlled study. The full verifier rejects it, but the converter accepts it and a fresh Proofpress receiver imports it as evidence. This is expected: intake validates its bounded projection, not the measurement calculation. It is not an additional fault or detection-rate observation. Finally, explicitly proposing a synthetic claim creates one candidate but still zero admissions and no governed context. No human approval or downstream agent run is exercised.

These observations demonstrate two non-equivalences: importable evidence is not necessarily verified evidence, and even an explicit candidate is not authorized context. They support interface separation, not the effectiveness of human governance. Historical TRACE schema validation is recorded separately; this new run tests conversion and intake, not upstream schema conformance.

V. RELATED WORK AND LIMITS

PROV-CONSTRAINTS already defines consistency conditions over provenance histories [2]. We do not claim that provenance or separation of concerns is itself new. Our concrete contribution is their composition around an agent handoff: an explicit decision record, executable task-specific checks, and a non-promoting evidence intake boundary. The RSI profile supplies paired-result recalculation, confirmation state and decision-to-submission identity; another domain would need its own checker.

in-toto verifies cryptographically linked software-supply-chain steps [3]; our unsigned packages provide no comparable authentication. MLflow records experiment parameters, metrics and artifacts [4]; a tracking system could retain the inputs our checks need. Neither system is an experimental baseline here. Storage, authentication, and decision-consistency checks address different parts of an evidence handoff.

Our results use one synthetic base, author-selected changes and shared checking components. The retained real-run case motivates capture but cannot be fully replayed. The integration demonstration is local and stops at an unadmitted candidate.

We have not measured capture completeness, human-review quality, audit effort, downstream agent benefit, cross-domain generalization or resistance to coordinated falsification. Reproducing an interval does not certify its validity under adaptive reuse of seeds. A rule-consistent decision can perform poorly on unseen games. Running checks outside the producing runtime does not authenticate the files that runtime supplies.

The Agent-Native Research Artifact companion [5] contains cases, claims, integration code, source pins and exploration history. Historical reward comparisons and a legacy reuse case remain there, not as additional verifier trials. Network identity, receipt-enforced admission and a complete multi-agent governance evaluation remain future work.

VI. CONCLUSION

Agent handoffs require more than retaining a process log. Capture–verify–govern separates what was submitted, what can be checked, and what a recipient may rely on. The RSI instantiation rejects six authored faults beyond format and file binding while accepting a consistent unsigned rewrite. Its TRACE integration demonstrates evidence intake without automatic claim admission. These bounded results support the separation of responsibilities, not validation of the entire framework: recorded, consistent and authorized are three different states.

REFERENCES

- [1] Aiming Lab, *RSI-Exam*, 2026. [Online]. Available: <https://github.com/aiming-lab/RSI-Exam>
- [2] J. Cheney, P. Missier, and L. Moreau, Eds., *Constraints of the PROV Data Model*, W3C Recommendation, Apr. 2013.
- [3] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, *in-toto: Providing farm-to-table guarantees for bits and bytes*, in *USENIX Security*, 2019, pp. 1393–1410.
- [4] MLflow, *ML Experiment Tracking*. [Online]. Available: <https://mlflow.org/docs/latest/ml/tracking/>
- [5] ARA Labs, *Agent-Native Research Artifact*. [Online]. Available: <https://github.com/ARA-Labs/Agent-Native-Research-Artifact>